

Diseño e Implementación de una Base de Datos para Telemetría de Videojuego y Evaluación de Experiencia de Usuario

Tercera Entrega

Beiker Romario Jaramillo Giraldo
Pontificia Universidad Javeriana
Bogotá, Colombia
jaramillo.beiker@javeriana.edu.co

Valentina Carrillo Peñuela
Pontificia Universidad Javeriana
Bogotá, Colombia
carrillo-valentina@javeriana.edu.co

Arturo Enrique Hincapié Orozco
Pontificia Universidad Javeriana
Bogotá, Colombia
arturo-hincapieo@javeriana.edu.co

Abstract—Este documento presenta la tercera entrega del proyecto semestral de Bases de Datos. Se implementaron consultas analíticas sobre la información de telemetría y experiencia de usuario, se evaluó el impacto de diferentes índices mediante EXPLAIN ANALYZE, se desarrollaron vistas para consultas frecuentes y se documentó un mecanismo para reconstruir completamente la base de datos y cargar los datos de prueba.

Index Terms—PostgreSQL, telemetría, UX, consultas analíticas, indexación, vistas materializadas.

I. INTRODUCCIÓN

Durante esta fase del proyecto se consolidó el diseño final de la base de datos, integrando el esquema relacional, el DDL y los mecanismos de reconstrucción necesarios para garantizar la reproducibilidad del sistema. Adicionalmente, se desarrollaron actividades orientadas al análisis de los datos de telemetría y experiencia de usuario recolectados desde Chocolate-Doom. Para ello, se implementaron consultas analíticas, se evaluó el impacto de diferentes índices sobre el rendimiento mediante pruebas de ejecución y se construyeron vistas y vistas materializadas con el fin de facilitar la reutilización de consultas frecuentes y optimizar futuros procesos de análisis.

II. DISEÑO DE LA BASE DE DATOS

A. Justificación del Diseño

El diseño de la base de datos se estructuró alrededor de dos fuentes principales de información: la telemetría generada por Chocolate-Doom y los datos de experiencia de usuario obtenidos mediante instrumentos UX. Se adoptó un modelo relacional normalizado para minimizar la redundancia y garantizar la integridad de los datos durante la carga y el análisis.

La tabla TelemetryEvent almacena los eventos registrados en cada tic de juego y se relaciona con las entidades Player, Game y Sector, permitiendo reconstruir trayectorias, analizar patrones de movimiento y estudiar la interacción entre jugadores. Por su parte, las tablas UXInstrument y UXResponse permiten registrar y evaluar la percepción de los usuarios sobre distintos aspectos de la experiencia de juego.

La separación de entidades como Episode, Game, Sector y Player facilita la reutilización de la información, evita

duplicidad de datos y permite realizar consultas analíticas complejas de manera eficiente. Además, la incorporación de restricciones de integridad, claves foráneas e índices garantiza la consistencia del modelo y mejora el rendimiento de las consultas sobre grandes volúmenes de telemetría.

B. Diagrama Entidad-Relación

La Figura 1 muestra el diagrama entidad-relación que captura las entidades principales del dominio y sus cardinalidades.

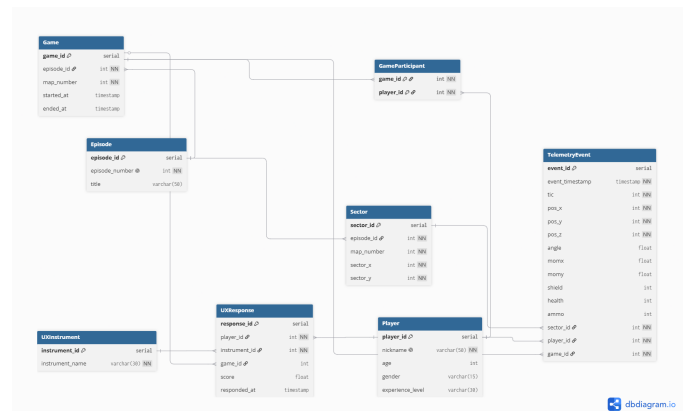


Fig. 1. Diagrama Entidad-Relación del sistema de telemetría y UX

C. Esquema Relacional

El siguiente código define las tablas principales del esquema con sus claves primarias, foráneas y restricciones de integridad.

```
Table Player {  
  player_id serial [pk, increment]  
  nickname varchar(50) [unique, not null]  
  age int [note: 'CHECK (age > 0)']  
  gender varchar(15)  
  experience_level varchar(30)  
}
```

```
Table Episode {
```

```

episode_id serial [pk, increment]
episode_number int [unique, not null]
title varchar(50)
}

Table Game {
game_id serial [pk, increment]
episode_id int [not null]
map_number int [not null]
started_at timestamp
ended_at timestamp
}

Table GameParticipant {
game_id int [not null]
player_id int [not null]

Indexes {
(game_id, player_id) [pk]
}
}

Table Sector {
sector_id serial [pk, increment]
episode_id int [not null]
map_number int [not null]
sector_x int [not null]
sector_y int [not null]

Indexes {
(episode_id, map_number, sector_x,
sector_y) [unique]
}
}

Table TelemetryEvent {
event_id serial [pk, increment]
event_timestamp timestamp [not null]
tic int [not null, note: 'CHECK (tic >= 0)']
pos_x int [not null]
pos_y int [not null]
pos_z int [not null]
angle float [note: 'CHECK (angle >= 0 AND
angle <= 360)']
momx float
momy float
shield int [note: 'CHECK (shield >= 0)']
health int [note: 'CHECK (health >= 0)']
ammo int [note: 'CHECK (ammo >= 0)']
sector_id int [not null]
player_id int [not null]
game_id int [not null]
}

Table UXInstrument {
instrument_id serial [pk, increment]
instrument_name varchar(30) [not null]
}

Table UXResponse {
response_id serial [pk, increment]
player_id int [not null]
instrument_id int [not null]
game_id int
score float
responded_at timestamp [default: `
CURRENT_TIMESTAMP`]

```

```

}

Ref: Game.episode_id > Episode.episode_id
Ref: GameParticipant.game_id > Game.game_id [
delete: cascade]
Ref: GameParticipant.player_id > Player.
player_id [delete: cascade]
Ref: Sector.episode_id > Episode.episode_id
Ref: TelemetryEvent.sector_id > Sector.
sector_id
Ref: TelemetryEvent.player_id > Player.
player_id
Ref: TelemetryEvent.game_id > Game.game_id
Ref: UXResponse.player_id > Player.player_id
Ref: UXResponse.instrument_id > UXInstrument.
instrument_id
Ref: UXResponse.game_id > Game.game_id

```

D. DDL

A continuación se presenta el código de definición de datos que implementa el esquema relacional en PostgreSQL, incluyendo la creación de todas las tablas con sus claves primarias, claves foráneas y restricciones de integridad.

```

CREATE TABLE Player (
player_id SERIAL PRIMARY KEY,
nickname VARCHAR(50) UNIQUE NOT NULL,
age INT CHECK (age > 0),
gender VARCHAR(15),
experience_level VARCHAR(30)
);

CREATE TABLE Episode (
episode_id SERIAL PRIMARY KEY,
episode_number INT UNIQUE NOT NULL,
title VARCHAR(50)
);

CREATE TABLE Game (
game_id SERIAL PRIMARY KEY,
episode_id INT NOT NULL,
map_number INT NOT NULL,
started_at TIMESTAMP,
ended_at TIMESTAMP,
CONSTRAINT fk_game_episode FOREIGN KEY (
episode_id)
REFERENCES Episode(episode_id)
);

CREATE TABLE GameParticipant (
game_id INT NOT NULL,
player_id INT NOT NULL,
PRIMARY KEY (game_id, player_id),
CONSTRAINT fk_gp_game FOREIGN KEY (game_id)
REFERENCES Game(game_id) ON DELETE
CASCADE,
CONSTRAINT fk_gp_player FOREIGN KEY (
player_id)
REFERENCES Player(player_id) ON DELETE
CASCADE
);

CREATE TABLE Sector (
sector_id SERIAL PRIMARY KEY,

```

```

episode_id INT NOT NULL,
map_number INT NOT NULL,
sector_x INT NOT NULL,
sector_y INT NOT NULL,
UNIQUE (episode_id, map_number, sector_x,
        sector_y),
CONSTRAINT fk_sector_episode FOREIGN KEY (
    episode_id)
    REFERENCES Episode(episode_id)
);

CREATE TABLE TelemetryEvent (
    event_id SERIAL PRIMARY KEY,
    event_timestamp TIMESTAMP NOT NULL,
    tic INT NOT NULL CHECK (tic >= 0),
    pos_x INT NOT NULL,
    pos_y INT NOT NULL,
    pos_z INT NOT NULL,
    angle FLOAT CHECK (angle >= 0 AND angle <=
        360),
    momx FLOAT,
    momy FLOAT,
    shield INT CHECK (shield >= 0),
    health INT CHECK (health >= 0),
    ammo INT CHECK (ammo >= 0),
    sector_id INT NOT NULL,
    player_id INT NOT NULL,
    game_id INT NOT NULL,
    CONSTRAINT fk_te_sector FOREIGN KEY (
        sector_id)
        REFERENCES Sector(sector_id),
    CONSTRAINT fk_te_player FOREIGN KEY (
        player_id)
        REFERENCES Player(player_id),
    CONSTRAINT fk_te_game FOREIGN KEY (game_id
    )
        REFERENCES Game(game_id)
);

CREATE TABLE UXInstrument (
    instrument_id SERIAL PRIMARY KEY,
    instrument_name VARCHAR(30) NOT NULL
);

CREATE TABLE UXResponse (
    response_id SERIAL PRIMARY KEY,
    player_id INT NOT NULL,
    instrument_id INT NOT NULL,
    game_id INT,
    score FLOAT,
    responded_at TIMESTAMP DEFAULT
        CURRENT_TIMESTAMP,
    CONSTRAINT fk_ux_player FOREIGN KEY (
        player_id)
        REFERENCES Player(player_id),
    CONSTRAINT fk_ux_instrument FOREIGN KEY (
        instrument_id)
        REFERENCES UXInstrument(instrument_id)
    ,
    CONSTRAINT fk_ux_game FOREIGN KEY (game_id
    )
        REFERENCES Game(game_id)
);

```

III. CONSULTAS ANALÍTICAS

Se implementaron seis consultas analíticas basadas en los requerimientos establecidos en el proyecto.

A. Consulta 1: Sectores y Frecuencia de Movimientos

En esta consulta se analiza el comportamiento espacial de los jugadores identificando los sectores más transitados a partir de los datos reales de telemetría. Adicionalmente, se promedia el estado de salud y escudo para determinar las zonas de mayor peligro o conflicto en el mapa.

```

SELECT
    s.sector_id,
    e.title AS episodio,
    s.map_number AS mapa,
    s.sector_x,
    s.sector_y,
    COUNT(t.event_id) AS
        total_movimientos_tics,
    ROUND(AVG(t.shield)::numeric, 2) AS
        escudo_promedio,
    ROUND(AVG(t.health)::numeric, 2) AS
        salud_promedio
FROM TelemetryEvent t
JOIN Sector s ON t.sector_id = s.sector_id
JOIN Episode e ON s.episode_id = e.episode_id
GROUP BY s.sector_id, e.title, s.map_number,
        s.sector_x, s.sector_y
ORDER BY total_movimientos_tics DESC
LIMIT 5;

```

sector_id	episodio	mapa	sector_x	sector_y	total_movimientos_tics	escudo_promedio_en_sector	immersion_espaio
1678	Thy Flesh Consumed	5	6	0	37	5.51	
603	The Shores of Hell	4	6	0	18	5.72	
1669	Thy Flesh Consumed	5	5	0	14	4.93	
1615	Thy Flesh Consumed	5	0	0	14	2.86	
649	The Shores of Hell	5	-8	-7	12	24.42	

Fig. 2. Resultado de la consulta 1: Sectores más transitados en el mapa de calor

B. Consulta 2: Número de Tics Donde los Jugadores Estuvieron Juntos

Esta consulta evalúa la co-presencia entre jugadores, contabilizando los tics en que dos jugadores distintos compartieron el mismo sector durante la misma partida.

```

SELECT
    t1.player_id AS jugador_1,
    t2.player_id AS jugador_2,
    COUNT(DISTINCT t1.tic) AS tics_juntos
FROM TelemetryEvent t1
JOIN TelemetryEvent t2
    ON t1.game_id = t2.game_id
    AND t1.tic = t2.tic
    AND t1.sector_id = t2.sector_id
    AND t1.player_id < t2.player_id
GROUP BY t1.player_id, t2.player_id
ORDER BY tics_juntos DESC;

```

	jugador_1 integer	jugador_2 integer	tics_juntos bigint
1	1	3	278
2	5	6	248
3	4	5	244
4	1	2	241
5	2	6	216
6	1	5	216
7	1	4	215
8	2	4	214
9	3	4	182
10	3	5	175

Fig. 3. Resultado de la consulta 2: Número de tics donde los jugadores estuvieron juntos

C. Consulta 3: Promedio UX de Jugadores con Trayectorias Superiores al Promedio

Esta consulta relaciona los datos de telemetría con los resultados de experiencia de usuario obtenidos mediante el instrumento PENS. Primero se calcula la duración de las trayectorias de cada jugador y se identifica a aquellos cuya actividad supera el promedio general.

```
WITH duraciones AS (
    SELECT
        player_id,
        game_id,
        MAX(tic) - MIN(tic) AS duracion
    FROM TelemetryEvent
    GROUP BY player_id, game_id
),
promedio AS (
    SELECT AVG(duracion) AS prom
    FROM duraciones
),
ux_promedio AS (
    SELECT
        player_id,
        AVG(score) AS score_promedio
    FROM UXResponse
    GROUP BY player_id
)
SELECT
    p.nickname,
    u.score_promedio
FROM duraciones d
JOIN promedio pr ON d.duracion > pr.prom
JOIN ux_promedio u ON d.player_id = u.
    player_id
JOIN Player p ON p.player_id = d.player_id
GROUP BY p.nickname, u.score_promedio
```

```
ORDER BY u.score_promedio DESC;
```

	nickname character varying (50)	score_promedio double precision
1	David	6.190476190476191
2	Beiker	5.476190476190476
3	Ana	5.142857142857143
4	Valentina	4.666666666666667
5	Arturo	4.0476190476190474
6	Lisa	3.3333333333333335

Fig. 4. Resultado de la consulta 3: Promedio UX de jugadores con trayectorias superiores al promedio

D. Consulta 4: Duración Promedio de Partidas por Mapa

Esta consulta permite analizar el tiempo promedio que los jugadores permanecen en cada mapa de los distintos episodios. Utilizando las marcas temporales de inicio y finalización de cada sesión, se calcula la duración promedio de las partidas.

```
SELECT
    e.title AS episodio,
    g.map_number AS mapa,
    COUNT(g.game_id) AS total_partidas_jugadas
    ,
    TO_CHAR(AVG(g.ended_at - g.started_at), '
        MI:SS')
        AS duracion_promedio_min_seg,
    ROUND(AVG(EXTRACT(EPOCH FROM
        (g.ended_at - g.started_at)) / 60)::
        numeric, 2)
        AS duracion_promedio_minutos_decimal
FROM Game g
JOIN Episode e ON g.episode_id = e.episode_id
GROUP BY e.title, g.map_number
ORDER BY e.title, g.map_number;
```

episodio character varying (50)	mapa integer	total_partidas_jugadas bigint	duracion_promedio_min_seg text	duracion_promedio_minutos_decimal numeric
1	Knee-Deep in the Dead	4	13 11:50	11.85
2	Knee-Deep in the Dead	5	20 11:39	11.65
3	The Inferno	4	13 12:23	12.38
4	The Inferno	5	19 12:00	12.00
5	The Shores of Hell	4	20 11:57	11.95
6	The Shores of Hell	5	13 11:50	11.85
7	Thy Flesh Consumed	4	17 11:21	11.35

Fig. 5. Resultado de la consulta 4: Duración promedio de partidas por mapa

E. Consulta 5: Distancia Total Recorrida y Velocidad Promedio por Jugador

Esta consulta reconstruye las trayectorias de movimiento de cada jugador utilizando las coordenadas espaciales registradas en la telemetría. A través de funciones ventana se comparan

posiciones consecutivas para calcular la distancia recorrida entre tics.

```
WITH trayectoria AS (
  SELECT
    player_id, game_id, tic,
    pos_x, pos_y, pos_z,
    LAG(pos_x) OVER (
      PARTITION BY player_id, game_id
      ORDER BY tic
    ) AS prev_x,
    LAG(pos_y) OVER (
      PARTITION BY player_id, game_id
      ORDER BY tic
    ) AS prev_y,
    LAG(pos_z) OVER (
      PARTITION BY player_id, game_id
      ORDER BY tic
    ) AS prev_z
  FROM TelemetryEvent
)
SELECT
  player_id,
  ROUND(SUM(SQRT(
    POWER(pos_x - prev_x, 2) +
    POWER(pos_y - prev_y, 2) +
    POWER(pos_z - prev_z, 2)
  ))::numeric, 2) AS distancia_total,
  ROUND(AVG(SQRT(
    POWER(pos_x - prev_x, 2) +
    POWER(pos_y - prev_y, 2) +
    POWER(pos_z - prev_z, 2)
  ))::numeric, 2) AS velocidad_promedio
FROM trayectoria
WHERE prev_x IS NOT NULL
GROUP BY player_id
ORDER BY distancia_total DESC;
```

	player_id integer	distancia_total numeric	velocidad_promedio numeric
1	3	1924237.14	266.74
2	5	1814730.42	275.54
3	1	1721928.48	235.62
4	4	1610778.59	217.23
5	2	1525702.92	208.06
6	6	1195243.26	179.52

Fig. 6. Resultado de la consulta 5: Distancia total y velocidad promedio por jugador

F. Consulta 6: Jugadores con Mayor Proximidad Promedio

Esta consulta analiza el nivel de proximidad espacial entre pares de jugadores durante las partidas. Para ello, compara las posiciones registradas en un mismo tic y dentro de una misma sesión de juego, calculando la distancia euclidiana entre los participantes.

```
SELECT
```

```
t1.player_id AS jugador_1,
t2.player_id AS jugador_2,
ROUND(AVG(SQRT(
  POWER(t1.pos_x - t2.pos_x, 2) +
  POWER(t1.pos_y - t2.pos_y, 2) +
  POWER(t1.pos_z - t2.pos_z, 2)
))::numeric, 2) AS distancia_promedio
FROM TelemetryEvent t1
JOIN TelemetryEvent t2
  ON t1.game_id = t2.game_id
  AND t1.tic = t2.tic
  AND t1.player_id < t2.player_id
GROUP BY t1.player_id, t2.player_id
ORDER BY distancia_promedio ASC
LIMIT 10;
```

	jugador_1 integer	jugador_2 integer	distancia_promedio numeric
1	1	3	10.41
2	5	6	10.50
3	3	4	10.64
4	3	5	10.77
5	1	2	10.83
6	1	6	10.85
7	2	4	10.86
8	3	6	11.00
9	1	5	11.08
10	2	6	11.18

Fig. 7. Resultado de la consulta 6: Jugadores con mayor proximidad promedio

IV. EVALUACIÓN DE ÍNDICES

Se implementaron tres índices para optimizar consultas frecuentes sobre la tabla TelemetryEvent.

A. Índice 1: (game_id, player_id)

```
CREATE INDEX idx_telemetry_game_player
  ON TelemetryEvent (game_id, player_id);

EXPLAIN ANALYZE
SELECT event_timestamp, pos_x, pos_y, health
FROM TelemetryEvent
WHERE game_id = 5 AND player_id = 4;
```

	QUERY PLAN
	text
1	Seq Scan on telemetryevent (cost=0.00..1708.49 rows=73 width=20) (actual time=0.068..2.643 rows=113.00 loops=1)
2	Filter: ((game_id = 5) AND (player_id = 4))
3	Rows Removed by Filter: 42920
4	Buffers: shared hit=1063
5	Planning Time: 0.084 ms
6	Execution Time: 2.664 ms
Total rows: 6 Query complete 00:00:00.051	

Fig. 8. EXPLAIN ANALYZE antes del índice 1

	QUERY PLAN
	text
1	Bitmap Heap Scan on telemetryevent (cost=5.04..235.09 rows=73 width=20) (actual time=0.030..0.043 rows=113.00 loops=1)
2	Recheck Cond: ((game_id = 5) AND (player_id = 4))
3	Heap Blocks: exact=4
4	Buffers: shared hit=6
5	-> Bitmap Index Scan on idx_telemetry_game_player (cost=0.00..5.02 rows=73 width=0) (actual time=0.018..0.018 rows=113.00 loops=1)
6	Index Cond: ((game_id = 5) AND (player_id = 4))
7	Index Searches: 1
8	Buffers: shared hit=2
9	Planning Time: 0.083 ms
10	Execution Time: 0.060 ms
Total rows: 10 Query complete 00:00:00.076	

Fig. 9. EXPLAIN ANALYZE después del índice 1

Discusión: Este índice compuesto eliminó el escaneo secuencial masivo de 2.664 ms, transformándolo en un escaneo indexado directo de apenas 0.060 ms. La búsqueda pasó de milisegundos enteros a microsegundos gracias a que el motor redujo drásticamente las páginas leídas.

B. Índice 2: event_timestamp

```
CREATE INDEX idx_telemetry_timestamp
ON TelemetryEvent (event_timestamp);

EXPLAIN ANALYZE
SELECT tic, player_id, health
FROM TelemetryEvent
WHERE event_timestamp
BETWEEN '2026-06-01 18:15:00'
AND '2026-06-01 18:30:00';
```

	QUERY PLAN
	text
1	Seq Scan on telemetryevent (cost=0.00..1708.49 rows=417 width=12) (actual time=0.095..1.684 rows=357.00 loops=1)
2	Filter: ((event_timestamp >= '2026-06-01 18:15:00':timestamp without time zone) AND (event_timestamp <= '2026-06-01 18:30:00':timestamp without time zone))
3	Rows Removed by Filter: 42676
4	Buffers: shared hit=1063
5	Planning Time: 0.166 ms
6	Execution Time: 1.718 ms

Fig. 10. EXPLAIN ANALYZE antes del índice 2

	QUERY PLAN
	text
1	Bitmap Heap Scan on telemetryevent (cost=8.56..810.90 rows=417 width=12) (actual time=0.032..0.089 rows=357.00 loops=1)
2	Recheck Cond: ((event_timestamp >= '2026-06-01 18:15:00':timestamp without time zone) AND (event_timestamp <= '2026-06-01 18:30:00':timestamp without time zone))
3	Heap Blocks: exact=12
4	Buffers: shared hit=14
5	-> Bitmap Index Scan on idx_telemetry_timestamp (cost=0.00..8.46 rows=417 width=0) (actual time=0.015..0.015 rows=357.00 loops=1)
6	Index Cond: ((event_timestamp >= '2026-06-01 18:15:00':timestamp without time zone) AND (event_timestamp <= '2026-06-01 18:30:00':timestamp without time zone))
7	Index Searches: 1
8	Buffers: shared hit=2
9	Planning:
10	Buffers: shared hit=16 read=3
11	Planning Time: 0.954 ms
12	Execution Time: 0.116 ms

Fig. 11. EXPLAIN ANALYZE después del índice 2

Discusión: Al indexar cronológicamente la tabla, las consultas por series de tiempo y ventanas de juego dejaron de recorrer la telemetría de punta a punta. El motor ahora localiza de inmediato el rango de tics consultado dentro del árbol.

C. Índice 3: (episode_id, map_number) en Sector

```
CREATE INDEX idx_sector_lookup
ON Sector (episode_id, map_number);
```

```
EXPLAIN ANALYZE
SELECT sector_id, sector_x, sector_y
FROM Sector
WHERE episode_id = 1 AND map_number = 5;
```

	QUERY PLAN
	text
1	Bitmap Heap Scan on sector (cost=10.48..24.71 rows=215 width=12) (actual time=0.069..0.120 rows=215.00 loops=1)
2	Recheck Cond: ((episode_id = 1) AND (map_number = 5))
3	Heap Blocks: exact=2
4	Buffers: shared hit=5
5	-> Bitmap Index Scan on idx_sector_episode_id_map_number_sector_x_sector_y_key (cost=0.00..10.43 rows=215 width=0) (actual time=0.039..0.039 rows=215.00 loops=1)
6	Index Cond: ((episode_id = 1) AND (map_number = 5))
7	Index Searches: 1
8	Buffers: shared hit=3
9	Planning:
10	Buffers: shared hit=5
11	Planning Time: 0.664 ms
12	Execution Time: 0.164 ms

Fig. 12. EXPLAIN ANALYZE antes del índice 3

	QUERY PLAN
	text
1	Bitmap Heap Scan on sector (cost=6.48..20.71 rows=215 width=12) (actual time=0.049..0.065 rows=215.00 loops=1)
2	Recheck Cond: ((episode_id = 1) AND (map_number = 5))
3	Heap Blocks: exact=2
4	Buffers: shared hit=2 read=2
5	-> Bitmap Index Scan on idx_sector_buscar (cost=0.00..6.43 rows=215 width=0) (actual time=0.024..0.024 rows=215.00 loops=1)
6	Index Cond: ((episode_id = 1) AND (map_number = 5))
7	Index Searches: 1
8	Buffers: shared read=2
9	Planning:
10	Buffers: shared hit=18 read=1
11	Planning Time: 1.073 ms
12	Execution Time: 0.084 ms

Fig. 13. EXPLAIN ANALYZE después del índice 3

Discusión: Este índice personalizado demostró ser más veloz que el índice automático generado por la restricción UNIQUE. Al almacenar únicamente las dos columnas del filtro, procesó una estructura mucho más ligera y compacta en memoria, reduciendo el costo estimado.

V. VISTAS

A. Vista 1: Resumen de Actividad por Jugador

Esta vista resume la actividad de cada jugador, incluyendo el número de partidas disputadas y su puntaje UX promedio.

```
CREATE VIEW v_actividad_jugador AS
SELECT
    p.player_id,
    p.nickname,
    p.experience_level,
    COUNT(DISTINCT gp.game_id) AS
        total_partidas,
    ROUND(AVG(ux.score)::numeric, 2) AS
        score_ux_promedio
FROM Player p
JOIN GameParticipant gp ON p.player_id = gp.
    player_id
LEFT JOIN UXResponse ux ON p.player_id = ux.
    player_id
GROUP BY p.player_id, p.nickname, p.
    experience_level;
```

B. Vista 2: Actividad por Sector

Esta vista consolida la actividad registrada en cada sector del mapa, facilitando el análisis espacial de la telemetría.

```
CREATE VIEW v_sector_activity AS
SELECT
    s.sector_id,
    e.title AS episodio,
    s.map_number,
    s.sector_x,
    s.sector_y,
    COUNT(t.event_id) AS total_eventos,
    ROUND(AVG(t.health)::numeric, 2) AS
        salud_promedio,
    ROUND(AVG(t.shield)::numeric, 2) AS
        escudo_promedio
FROM TelemetryEvent t
JOIN Sector s ON t.sector_id = s.sector_id
JOIN Episode e ON s.episode_id = e.episode_id
GROUP BY s.sector_id, e.title, s.map_number,
    s.sector_x, s.sector_y;
```

VI. VISTA MATERIALIZADA

Debido al gran volumen de registros almacenados en TelemetryEvent, esta vista materializada almacena de forma persistente los principales indicadores de actividad por sector, evitando recalcular agregaciones costosas en consultas frecuentes.

```
CREATE MATERIALIZED VIEW mv_hotspots AS
SELECT
    s.sector_id,
    e.title AS episodio,
    s.map_number,
    s.sector_x,
    s.sector_y,
    COUNT(t.event_id) AS total_eventos,
    ROUND(AVG(t.health)::numeric, 2) AS
        salud_promedio,
    ROUND(AVG(t.shield)::numeric, 2) AS
        escudo_promedio
```

```
FROM TelemetryEvent t
JOIN Sector s ON t.sector_id = s.sector_id
JOIN Episode e ON s.episode_id = e.episode_id
GROUP BY s.sector_id, e.title, s.map_number,
    s.sector_x, s.sector_y;
```

VII. MECANISMO DE RECONSTRUCCIÓN

Para garantizar la reproducibilidad del proyecto se implementó un script encargado de reconstruir completamente la base de datos. El archivo rebuild.sql ejecuta secuencialmente la creación del esquema relacional, la carga de los datos de muestra, la creación de índices y la generación de vistas y vistas materializadas. De esta forma, cualquier usuario con PostgreSQL puede reconstruir completamente la base de datos y reproducir los análisis presentados en este trabajo. Se debe tener en cuenta que la ruta /ruta/al/proyecto/ debe reemplazarse por la ubicación real donde se encuentren los archivos del proyecto.

```
\i '/ruta/al/proyecto/ddl.sql'
\copy Player FROM '/ruta/al/proyecto/
    TablePlayer.csv'
    CSV HEADER
\copy Episode FROM '/ruta/al/proyecto/
    TableEpisode.csv'
    CSV HEADER
\copy Game FROM '/ruta/al/proyecto/TableGame.
    csv'
    CSV HEADER
\copy GameParticipant FROM
    '/ruta/al/proyecto/TableGameParticipant.
    csv'
    CSV HEADER
\copy Sector FROM '/ruta/al/proyecto/
    TableSector.csv'
    CSV HEADER
\copy TelemetryEvent FROM
    '/ruta/al/proyecto/TableTelemetryevent.
    csv'
    CSV HEADER
\copy UXInstrument FROM
    '/ruta/al/proyecto/TableUxinstrument.csv'
    CSV HEADER
\copy UXResponse FROM
    '/ruta/al/proyecto/TableUxresponse.csv'
    CSV HEADER
\i '/ruta/al/proyecto/indices.sql'
\i '/ruta/al/proyecto/vistas.sql'
\i '/ruta/al/proyecto/materialized_view.sql'
```

VIII. CONCLUSIONES

La implementación de consultas analíticas permitió extraer información relevante sobre el comportamiento de los jugadores y su experiencia de usuario. Los índices mejoraron significativamente el rendimiento de las consultas más frecuentes, mientras que las vistas y vistas materializadas facilitaron la reutilización de análisis complejos. Finalmente, el mecanismo de reconstrucción garantiza la reproducibilidad completa del proyecto.

REFERENCES

- [1] A. O. Calderon, “trajectory_doom,” GitHub repository, 2024. [Online]. Available: https://github.com/aocalderon/trajectory_doom. [Accessed: 03-Jun-2026].